

# **Linux Programming: Assignment-9:**

## **20-09-2025**

**NAME-** G JYOTHSNA

**ROLL NO-** 04

**USN-** ENG24CY0021

**CLASS-** CYBER SECURITY

**SECTION-** C

**21. Write a shell script using if...else to check if a number is even or odd.**

Ans:

```
#!/bin/bash
read -p "Enter a number: " num
if [ $((num % 2)) -eq 0 ]; then
    echo "$num is even."
else
    echo "$num is odd."
fi
```

Output:

If the user enter 0

\$ ./even\_odd.sh

Enter a number: 0

0 is even

**22. Explain the difference between if and case statements in bash.**

Ans:

If statement

If statement in shell is used to test condition

It executes a block of code if a condition is true and otherwise optionally executes else or elif

It is usually used when you comparisons or complex condition you want to execute all the condition

Case statement:

This statement is used to match a value against patterns

Ideal when you have multiple different option to choose from

It is used when you have check strings

It uses case...esac

**23. Write a script to find the largest of three numbers entered by the user.**

Ans:

```
#!/bin/bash
```

```
read -p "Enter first number: " num1
read -p "Enter second number: " num2
read -p "Enter third number: " num3
```

```
# Compare
```

```
if [ $num1 -ge $num2 ] && [ $num1 -ge $num3 ]; then
    echo "The largest number is $num1"
elif [ $num2 -ge $num1 ] && [ $num2 -ge $num3 ]; then
    echo "The largest number is $num2"
else
    echo "The largest number is $num3"
fi
```

Output:

```
Enter first number: 10
Enter second number: 25
Enter third number: 15
The largest number is 2
```

**24. How do you use a for loop to traverse an array in bash?**

**Give an example. The array is defined as arr=(123, "Abs", -2.3, 'A', 23.56, 0).**

Ans:

```
#!/bin/bash
# Define the array
arr=(123 "Abs" -2.3 'A' 23.56 0)
# Traversing
for element in "${arr[@]}"
do
```

```
    echo "$element"  
done
```

Output:

```
123  
Abs  
-2.3  
A  
23.56  
0
```

**25. Write a shell script to loop through all files in the current directory and display their names.**

Ans:

```
#!/bin/bash  
# Loop  
for file in *  
do  
    if [ -f "$file" ]; then  
        echo "$file"  
    fi  
done
```

Output:

```
file1.txt  
script.sh  
Notes.doc
```

**26. What is the difference between while and until loops in bash?**

Ans:

While loop:

This loop executes a block of code as long as the condition is true

If the condition is checked before each iteration hence while loop is also known as pre test loop

Syntax:

```
while [ condition ]  
do  
    # commands  
done
```

Until loop:

This loop executes a block of code as long as the condition is false

Stops the execution or terminates when the condition is true

Syntax:

```
until [ condition ]  
do  
    # commands  
done
```

## **27. Write a countdown timer script using a while loop.**

Ans:

```
#!/bin/bash  
read -p "Enter countdown time in seconds: " timer  
while [ $timer -gt 0 ]  
do  
    echo "Time remaining: $timer seconds"  
    sleep 1    # Wait for 1 second
```

```
((timer--)) # Decrease timer by 1  
done
```

```
echo "Time's up!"
```

Output:

If the user enters 4:

Enter countdown time in seconds: 5

Time remaining: 5 seconds

Time remaining: 4 seconds

Time remaining: 3 seconds

Time remaining: 2 seconds

Time remaining: 1 seconds

Time's up

## **28. How do you use break and continue statements in loops? Give examples.**

Ans:

Break statement is used to break the current loop and exit from that current loop

It is useful when you want to stop looping based on a condition

Example:

```
#!/bin/bash
```

```
for i in {1..10}
```

```
do
```

```
    if [ $i -eq 5 ]; then
```

```
        echo "Number 5 found, stopping the loop."
```

```
        break
```

```
    fi
```

```
    echo "Current number: $i"
```

done

Output:

Current number: 1

Current number: 2

Current number: 3

Current number: 4

Number 5 found, stopping the loop.

Continue statement:

This skips the iteration if the loop has any and moves to the next iteration

It is useful when you want to ignore certain values without stopping the loop

Example:

```
#!/bin/bash
```

```
for i in {1..10}
do
    if [ $i -eq 5 ]; then
        echo "Skipping number 5"
        continue
    fi
    echo "Current number: $i"
done
```

Output:

Current number: 1

Current number: 2

Current number: 3

Current number: 4  
Skipping number 5  
Current number: 6  
Current number: 7  
Current number: 8  
Current number: 9  
Current number: 10

**29. Write a script to check if a file exists or not using the if and else loop.**

Ans:

```
#!/bin/bash
read -p "Enter the file name to check: " filename
# Check if the file exists
if [ -e "$filename" ]; then
    echo "File '$filename' exists."
else
    echo "File '$filename' does not exist."
fi
```

**30. Write a script to calculate factorial of a number using for loop.**

Ans:

```
#!/bin/bash

# Read number from the user
read -p "Enter a number: " num

# Initialize factorial
factorial=1
```

```
# Calculate factorial using for loop
```

```
for (( i=1; i<=num; i++ ))
```

```
do
```

```
    factorial=$((factorial * i))
```

```
done
```

```
# Display the result
```

```
echo "Factorial of $num is $factorial"
```